

Preserving Operation Effects Relation in Group Editors

Du Li and Rui Li
Department of Computer Science
Texas A&M University
College Station, TX 77843-3112 USA
Email Contact: lidu@cs.tamu.edu

ABSTRACT

Consistency maintenance is a critical and challenging issue in many interactive groupware applications that can be modeled as group editors. This paper addresses two open consistency problems: divergence and operation effects relation violation. While the former has been partly solved, the latter has never been addressed in any published work. We propose a novel approach to solving both problems in the same framework. Particularly it preserves the effects relation of concurrent operations so that convergence is achieved automatically. Our work provides a new perspective into group editors and operational transformation algorithms.

Categories and Subject Descriptors

C.2.4 [Computer-Communication Networks]: Distributed Systems—*Distributed applications*; H.5.3 [Information Interfaces and Presentation]: Group and Organization Interfaces—*Collaborative computing*; *Synchronous interaction*

General Terms

Algorithms, Design, Human Factors

Keywords

CSCW, Collaborative Systems, Concurrency Control, Consistency, Operational Transformation, Group editors

1. INTRODUCTION

Group editors (e.g., [1, 7, 24]) are an important category of interactive groupware applications that allow a distributed group of people to work together on a set of shared data objects (document). They have been used frequently as a model and research vehicle for studying a wide range of groupware issues by numerous researchers since the birth of the CSCW field, to name but a few, [4, 7, 10, 12, 16, 26].

Over the past decade operational transformation (OT) [3, 7, 25] has become an established method for consistency

maintenance in group editors. Compared to alternative concurrency control methods such as locking and serialization, OT has been found uniquely promising in achieving convergence, causality and intention preservation without sacrificing responsiveness and concurrent work in Sun et al. [26].

The main property of OT, allowing users to edit any part of the shared data at any time, in particular matches the vision of Bentley and Dourish [2] of developing collaborative systems as a customizable collaboration medium. In light of their arguments, group editors with this property do not impose any constraint on how people use the technology. Users are allowed to use group editors for either asynchronous or synchronous editing. With the “right” single-user features (e.g., formatting) and multi-user features (e.g., [21, 22]), group editors appear capable of accommodating a variety of editing purposes, work styles, and processes [17].

In pursuit of these promises, perseverant efforts have been made since the seminal work of Ellis and Gibbs [7]. These have resulted in significant progress in the field, e.g., the distinction between transformation functions for different purposes [20, 26], the solution of various consistency problems [18, 26, 22], the implementation of group undo [12, 22], the generalization to string, sequence, and tree based operations and data structures [1, 3, 26], and the transparent leveraging of full-fledged single-user editors and word processor [13, 24]. Despite these achievements, there are still many open issues and research opportunities, such as the maintenance of user intentions and semantic consistency [25].

This paper looks into two issues that have not been addressed adequately in the literature. Specifically, we will give scenarios that reveal two types of consistency problems: divergence, and operation effects relation violation (ERV). The former has been solved in [19, 26, 27] by constraining the transformation paths. The latter has drawn very little attention in previous work and thus no published work to our knowledge is able to solve it. The absence of a solution to these problems calls for a revisit to existing OT algorithms and consistency models.

This paper proposes a novel transformation based approach to preserving the correct operation effects relation in group editors. Due to its root in effects relation violation, the divergence problem is then solved automatically so that convergence is achieved in the presence of arbitrary transformation paths. Our work also establishes a new framework for interpreting OT and a new consistency model (causality, single-operation effects, and multi-operation effects relation preservation, or CSM) for evaluating OT algorithms. Due to limitations of current technologies, it only attempts to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

CSCW'04, November 6–10, 2004, Chicago, Illinois, USA.
Copyright 2004 ACM 1-58113-810-5/04/0011 ...\$5.00.

formalize syntactic consistency and leaves semantic consistency to the interpretation of end users. Group editors that ensure CSM effectively implement a reliable collaboration medium atop which high-level mechanisms (e.g., [21, 22]) can be provided to help users preserve their intentions and maintain semantic consistency.

The rest of this paper is structured as follows. Sec. 2 elaborates the two consistency problems. Sec. 3 presents our CSM consistency model. Our approach is then presented in sections 4 and 5. Related works are compared in Sec. 6. Then Sec. 7 summarizes contributions and future directions.

2. BACKGROUND AND OPEN PROBLEMS

For simplicity and space reasons, we assume the classic string model of [7]. The shared document in question is represented as a character string, the position of its first character being zero. Significant operations are normalized as $\text{insert/delete}(C, P)$, which inserts or deletes character C at position P in some document state. This simplification will help us focus on analyzing and solving the real problems. As shown in [1, 3, 26], it is easy to generalize this abstraction to more sophisticated operations and data structures.

The shared document is replicated at all sites in question. Operations after execution change the document from one state to another. To distinguish different document states at different sites, we use S_i^p to denote a document state, where subscript i stands for the site id and superscript p distinguishes different states. Any operation is defined (generated or executed) relative to a given document state. The document state on which an operation O is generated, the *generation state* of O , may be different from the document state on which O is executed, the *execution state* of O . To differentiate, we annotate the execution form of an operation with its definition state. Given operation O^S , we say that O (more precisely its position parameter P) is defined on S and S is its *definition state*, regardless of whether S is actually its generation or execution state.

In OT algorithms, the execution form of an operation is obtained by transforming it against concurrent operations that have been executed earlier [7]. As a result, the position parameter of the operation is often shifted to incorporate the effects of those concurrent operations. Sun et al. [26] distinguishes the different purposes of transformation functions and defines the concepts of inclusion transformation (IT) and exclusion transformation (ET). $\text{IT}(O_1^S, O_2^S)$ transforms an operation O_1^S against another operation O_2^S in such a way that the effect of O_2^S is effectively included. $\text{ET}(O_2^{S^{i+1}}, O_1^{S^i})$ transforms $O_2^{S^{i+1}}$ against $O_1^{S^i}$ in such a way that the effect of $O_1^{S^i}$ is effectively excluded from $O_2^{S^{i+1}}$, where S^{i+1} is the result of executing $O_1^{S^i}$ in S^i , or $S^i + [O_1^{S^i}] = S^{i+1}$.

Given three operations $O_1^{S^0}$, $O_2^{S^0}$ and $O_3^{S^0}$, $S^0 + [O_1^{S^0}] = S^1$ and $S^0 + [O_2^{S^0}] = S^{1'}$. Let $O_1^{S^{1'}} = \text{IT}(O_1^{S^0}, O_2^{S^0})$ and $O_2^{S^1} = \text{IT}(O_2^{S^0}, O_1^{S^0})$. The following two transformation properties, TP1 and TP2, are sufficient and necessary for OT algorithms to guarantee convergence along arbitrary transformation paths [18].

$$\begin{aligned} \text{TP1: } S^0 + [O_1^{S^0}, O_2^{S^1}] &= S^0 + [O_2^{S^0}, O_1^{S^{1'}}] \\ \text{TP2: } \text{IT}(\text{IT}(O_3^{S^0}, O_1^{S^0}), O_2^{S^1}) &= \text{IT}(\text{IT}(O_3^{S^0}, O_2^{S^0}), O_1^{S^{1'}}) \end{aligned}$$

Consider the scenario in Fig. 1. Three sites 1, 2, 3 start from the same initial document state, $S_1^0 = S_2^0 = S_3^0 =$

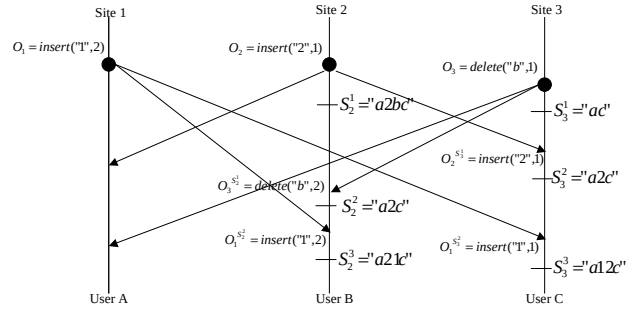


Figure 1: A scenario of divergence and ERV.

“abc”. They generate operations $O_1 = \text{insert}('1', 2)$, $O_2 = \text{insert}('2', 1)$, and $O_3 = \text{delete}('b', 1)$ concurrently, which result in $S_1^1 = \text{“ab1c”}$, $S_2^1 = \text{“a2bc”}$, and $S_3^1 = \text{“ac”}$, respectively. In the following we assume the IT functions in [22, 26].

At site 2, when O_3 is received, it is inclusively transformed against O_2 , resulting in $O_3^{S_2^1} = \text{delete}('b', 2)$. After $O_3^{S_2^1}$ is executed, the state becomes $S_2^2 = \text{“a2c”}$. When O_1 arrives, it is inclusively transformed against $[O_2^{S_2^0}, O_3^{S_2^1}]$, resulting in $O_1^{S_2^2} = \text{IT}(O_1^{S_1^0}, [O_2^{S_2^0}, O_3^{S_2^1}]) = \text{IT}(O_1^{S_1^0}, O_3^{S_2^1}) = \text{insert}('1', 2)$, whose execution leads to state $S_2^3 = \text{“a21c”}$.

At site 3, O_2 arrives first and is inclusively transformed against O_3 , resulting in $O_2^{S_3^1} = \text{insert}('2', 1)$. After $O_2^{S_3^1}$ is executed, the state becomes $S_3^2 = \text{“a2c”}$. When O_1 arrives, it is inclusively transformed first against O_3 , resulting in $O_1^{S_3^2} = \text{insert}('1', 1)$. Then $O_1^{S_3^2}$ is inclusively transformed against $O_2^{S_3^1}$. Since the site id of O_1 is smaller than that of O_2 , it is not shifted. Hence $O_1^{S_3^3} = \text{insert}('1', 1)$. After the execution of $O_1^{S_3^3}$, the state of site 3 becomes $S_3^3 = \text{“a12c”}$, which is not consistent with the state $S_2^3 = \text{“a21c”}$ of site 2.

Similar scenarios have been identified in [11, 20, 26]. As pointed out in [26], the problem cannot be fixed by simply reversing the site ids of O_1 and O_2 , because this quick fix works here but will fail in other similar scenarios.

The scenario in Fig. 1 actually reveals two consistency problems of different nature: First is divergence. After O_1 , O_2 and O_3 are executed at site 2 and site 3 in different orders (on the same initial state), the final states are not identical. The cause of this problem is that, after $O_1^{S_1^0}$ is inclusively transformed along different paths $[O_2^{S_2^0}, O_3^{S_2^1}]$ and $[O_3^{S_3^0}, O_2^{S_3^1}]$ at site 2 and site 3, respectively, different results are obtained, i.e., $O_1^{S_2^2} = \text{insert}('1', 2)$ and $O_1^{S_3^2} = \text{insert}('1', 1)$. That means the IT functions cannot satisfy the TP2 condition defined above. Hence we call this problem the TP2 puzzle.

The second problem is the operation effects relation violation (ERV) puzzle. If the landmark character ‘b’ had not been deleted, intuitively, the operation effect of O_2 (‘2’) should have been to the left of effect of O_1 (‘1’). However, the final state at site 3 obviously violates this correct effects relation. The confusions are even aggravated if the deleted landmark object is recovered by the selective undo mechanism of [22], which results in “a1b2c” at site 3 in Fig. 1.

The TP2 puzzle has been solved in previous work [19, 26, 27] by enforcing the same transformation path at all sites such that TP2 is no longer a necessary condition of convergence. However, it is still an open problem as how to achieve convergence in the presence of arbitrary transfor-

mation paths. Nevertheless, none of them can guarantee that the converged document will not violate the effects relation of concurrent operations. In scenarios like Fig. 1, it is equally possible for them to converge at “a12c” or “a21c”.

3. A CONSISTENCY MODEL

In Fig. 1, based on the landmark character ‘b’, it is obvious that the intended effect (or intention) of O_1 is to insert ‘1’ after ‘b’ and the intention of O_2 is to insert ‘2’ before ‘b’. Hence result “a21c” preserves the intentions of O_1 and O_2 while “a12c” does not. However, according to Sun et al. [26], both results are intention-preserving. Therefore it seems necessary to revisit the convergence, causality and intention-preservation (CCI) model defined in [26], or at least its intention-preservation part.

When processing $IT(O_1^{S_1^1}, O_2^{S_2^1})$ at site 3, if we had somehow known that O_2 was intended to insert a character to the left of O_1 ’s intended effect position relative to S^0 , we would have shifted the position parameter of $O_1^{S_1^1}$ to get the right result $O_3^{S_3^2} = \text{insert}(\text{“1”}, 2)$. Then the ERV puzzle would have been solved. As a result, all sites would have converged to the same effects relation preserving state, and then the TP2 puzzle would have not been a problem. The question is how to capture the intended effects relation.

3.1 Operation Effects Relation

Our intuition comes from the fact that the position of every character in a string is unique. That is, assuming that every appearance of a character has a unique identity, no two characters in a string take the same position. Therefore the position relation between all characters in any string is a total order. We denote such a total order as $\prec$. Apparently relation $\prec$ is transitive. For example, in the initial state $S^0 = \text{“abc”}$ of Fig. 1, we have ‘a’ $\prec$ ‘b’ $\prec$ ‘c’. Due to ‘b’ $\prec$ ‘1’ $\prec$ ‘c’ in state $S_1^1 = \text{“ab1c”}$ and ‘a’ $\prec$ ‘2’ $\prec$ ‘b’ in state $S_2^1 = \text{“a2bc”}$, we have ‘a’ $\prec$ ‘2’ $\prec$ ‘b’ $\prec$ ‘1’ $\prec$ ‘c’.

Characters in any document state are eventually a result of editing operations. Every characterwise operation O has an effect character $C(O)$. Hence we extend the total order $\prec$ of characters to operations. Given any two operations O_x and O_y , we say $O_x \prec O_y$, iff $C(O_x) \prec C(O_y)$. For example, in Fig. 1, we have $O_2 \prec O_3 \prec O_1$.

Note relation $\prec$ is not defined between any pair of inverse operations, O and O' , where O happened before O' (denoted as $O \rightarrow O'$)¹ and O' deletes the character inserted by O . Because the effect characters of inverse operations are the same, their position relation is not a total order.

However, the effects relation between concurrent operations is not always as obvious as in Fig. 1. For example, consider two operations, O_1 and O_2 , that are concurrent (denoted as $O_1 \parallel O_2$) and are both intended to insert their effect characters at the same position. In this case, we have to assume that either $O_1 \prec O_2$ or $O_2 \prec O_1$ is acceptable at the system level. Eventually some priority scheme has to be adopted to break the tie, as in [7].

3.2 Definition of the CSM Model

A group editor is CSM-consistent iff the following three conditions hold:

1. **Causality preservation:** all operations are executed in their cause-effect order. To verify, for any two operations O_1 and O_2 , if $O_1 \rightarrow O_2$, then O_1 is executed before O_2 at all sites.
2. **Single-operation effects preservation:** the effect of executing any operation in any execution state achieves the same effect as in its generation state. To verify, suppose operation O is generated in state S_i^p and executed in state S_j^u . If O is a deletion, the same character is deleted from both states; or if O is an insertion, the same total order maintains between any two characters that appear in both resulted states S_i^{p+1} and S_j^{u+1} .
3. **Multi-operation effects relation preservation:** the effects relation of any two operations maintains after they are executed in any states. To verify, given any two operations O_i and O_j that are generated in states S^i and S^j , respectively, without loss of generality, assume $C(O_i) \prec C(O_j)$. The same effects relation holds after they are both executed in any state S^k .

The above definitions of relation $\prec$ and consistency conditions as well as their verification methods can be easily extended to other data structures beyond strings, such as sequences [1] and trees [3], in which objects can also be uniquely identified by their positions in the documents.

3.3 Syntactic and Semantic Consistency

The CCI model [26] is well-accepted in group editors, e.g., [1, 24]. The causality preservation condition is inherited because it is generally required in distributed systems. Its convergence condition is no longer necessary because it is implied by the **S** and **M** conditions. It can be shown that, after all generated operations have been executed, if the **S** and **M** conditions are satisfied, all sites will have exactly the same set of characters and they are in exactly the same order. In other words, the states of all sites converge.

The intention preservation condition in Sun et al. [26] is defined as (1) that the effect of executing any operation O at remote sites achieves the same effect as executing O at the local site at the time of its generation, and (2) that the execution effects of concurrent operations do not interfere each other. The first condition is inherited in our notion of single-operation effects preservation. The second condition seems more general than our notion of multi-operation effects relation preservation but less formalized. It does not clearly specify exactly under which condition concurrent execution does or does not *interfere* each other.

In fact, concurrent operations do interfere each other unless “properly” coordinated. A classic example is that two users attempt to fix the grammatical error in sentence “There should be student.” at the same time, one trying to insert “a” before and another to insert “s” after “student”. Current group editors will produce a syntactically consistent but semantically inconsistent result “There should be a students” [26]. *Syntactic consistency* requires an identical view of the shared data, while *semantic consistency* demands that the converged view *make sense* in the application context [5, 25]. Apparently semantic consistency cannot be easily formalized and verified by pure system approaches.

Hence the CSM model reflects a more realistic approach to addressing consistency problems in group editors. It envisions a reliable collaboration medium that guarantees syn-

¹The definitions of relations $\rightarrow$ and $\parallel$ follow those of [7].

tactic consistency and deliberately leaves semantic consistency to the interpretation of end users. As an integrated and active part of the collaborative system, users will be able to maintain semantic consistency and preserve their intentions with further helps from, e.g., error recovery and locking mechanisms [21, 22], real-time communication and awareness [8], and coordination protocols [17].

The fact that group editors allow users to edit the same part of the shared document at the same time does not necessarily imply that users have to work that way. It makes perfect sense for users to work on different parts at the same time and take turns to work on the same part, e.g., by following social protocols. The question is how to “properly” coordinate individual editing activities. On one hand, formalizing protocols into the system is generally difficult and often suffers from the dangers of being inflexible [9]. On the other hand, a system that totally relies on the users’ conscious following of implicit protocols or processes may risk laying too much burden on its users, although significant flexibility can be achieved [6].

Argument about different coordination paradigms and implementation of specific coordination mechanisms are beyond the scope of this paper. However, the focus of group editors has been traditionally on supporting familiar small groups [7], in which people usually coordinate well by social protocols [17]. It makes sense and is often more effective in this context to rely on social protocols, which is able to easily accommodate a variety of work styles and writing processes while saving the costs and problems of formalization. It is generally assumed that, with real-time communication and awareness, users can easily discover and resolve conflicts and semantic inconsistencies in a timely manner [7, 8, 26].

4. TRANSFORMATION FUNCTIONS

We use operational transformation (OT) for achieving CSM consistency in group editors. For space reasons, the formal proof that our approach satisfies CSM is given in [14]. As a convention in OT [7, 26], our algorithm is divided into two parts: the IT/ET functions are defined in this section, and the control algorithm will be given in Sec. 5.

For responsiveness, local operations are always executed immediately once generated. Remote operations are (inclusively) transformed against concurrent operations that have been executed earlier and then the transformed results are executed in current state. Causality preservation is guaranteed in the control algorithm, e.g., by using the standard state vectors [7], such that a remote operation O is not executed until it is causally ready, i.e., every operation O_x , $O_x \rightarrow O$, has been executed.

4.1 Inclusion Transformation

If the correct relation $\prec$ between the effects of all concurrent operations is known, then we can always correctly execute remote operations while preserving single-operation effects and the multi-operation effects relation.

Fig. 2 defines a conceptual IT function. When O_1 is inclusively transformed O_2 , they must be defined in the same state and O_2 must have been executed [26]. Only when $O_2 \prec O_1$, i.e., the effect position of O_2 should be to the left of that of O_1 , do we need shift the position of O_1 . If O_2 is an insertion, we shift the position of O_1 by one position to the right; Or if O_2 is a deletion, we shift the position of O_1 by one position to the left. Function $T(O)$ returns the type

FUNC 1. $IT(O_1^S, O_2^S): O_1^{S'}, \text{ where } S' = S + O_2^S \text{ and } O_1 \parallel O_2.$

```

 $O_1^{S'} := O_1^S;$ 
if  $O_2 \prec O_1$ 
  if  $T(O_2) = \text{INS}$ 
     $P(O_1^{S'}) := P(O_1^{S'}) + 1;$ 
  else //  $T(O_2) = \text{DEL}$ 
     $P(O_1^{S'}) := P(O_1^{S'}) - 1;$ 
return  $O_1^{S'};$ 

```

Figure 2: IT reinterpreted by $\prec$.

(INS or DEL) of operation O , and $P(O)$ returns the effect position of O relative to its definition state.

4.2 Determining the Effects Relation

The problem is how to determine the effects relation $\prec$. Clearly due to the counterexample in Fig. 1 comparing the position parameters and site ids of operations alone is insufficient for this purpose. Unfortunately, almost all previous OT algorithms (except [11, 20]) are based on the IT functions of [7] which use position parameters to determine the effects relation and resort to site ids if there is a tie.

In Fig. 1, when processing $IT(O_1^{S_3^1}, O_2^{S_3^1})$ at site 3, because $P(O_1^{S_3^1}) = P(O_2^{S_3^1})$, the original IT functions use site ids to break the tie and get the wrong result. However, due to relation $O_2 \prec O_1$, we know that this is actually a false tie. The reason that previous work fails in this false-tie puzzle is that they commit prematurely to a priority scheme.

The comparison between $P(O_1^{S_3^1})$ and $P(O_2^{S_3^1})$ is based on a state in which a third operation O_3 has been executed. What if we go back to a prior state in which O_3 is not executed? The natural choice here is the initial state S^0 . Without the effect of O_3 , we know that the effect position of $O_1^{S^0}$ is 2 and that of $O_2^{S^0}$ is 1. That is, $O_2 \prec O_1$.

To extend this intuition into a general solution, however, we need to answer two questions. First, which previous state S' shall we examine when processing $IT(O_1^S, O_2^S)$ between any two concurrent operations O_1 and O_2 ? Second, how to calculate their effect positions relative to S' ?

Apparently S' must be a common previous state of both O_1^S and O_2^S . Otherwise, even after we calculate their effect positions relative to two different previous states, it makes no sense to compare them. Suppose that S^0 is the initial state at all sites, and O_1 and O_2 are generated in state S_i^p and state S_j^u , respectively. Then due to causality preservation, S_i^p and S_j^u must also be prior to current state S .

Let SQ be a sequence of n operations $O_1, O_2, \dots, O_n$, the execution of which in state S^1 results in state S^n , or $S^1 + SQ = S^n$. We denote the relation that S^1 is prior to S^n as $S^1 < S^n$ if $n > 0$. We say $S^1 \leq S^n$ if $n \geq 0$. Continuing with the above analysis, obviously we have $S^0 \leq S'$, $S' \leq S_i^p$, $S' \leq S_j^u$, $S_i^p \leq S$, and $S_j^u \leq S$, but S_i^p may not be equal to S_j^u . The relation between states is only a partial order.

Hence we do not really need to go back to state S^0 . Instead, a possibly later state S' that is “closest” to both S_i^p and S_j^u would suffice and save some computation. We introduce the concept of **last synchronization point** (LSP) to represent such a state S^{lsp} . Let V_1 and V_2 be the state vectors of O_1 and O_2 , respectively. Let V_{min} be a state vector, each element of which is equal to the minimal value of the corresponding elements of V_1 and V_2 . Then S^{lsp} is the state that corresponds to V_{min} .

We introduce the notation of $\beta_{S'}(O)$ to represent the effect position of operation O^S relative to any state S' that is prior to its definition state S . Suppose the effect positions of O_1 and O_2 relative to state S^{isp} be $\beta_{isp}(O_1)$ and $\beta_{isp}(O_2)$, respectively. The algorithm for computing β_{isp} will be given in Sec. 5.2. We define $O_1 \prec O_2$ in Fig. 3. It is easy to show that the original IT functions of [7] is a special case of our IT function (Fig. 2) together with this definition of the $\prec$ relation. We postpone the use of the priority scheme until both their β and P values are tied. Note that $P(O_1)$ and $P(O_2)$ are relative to the current state S , that is, the state in which $IT(O_1^S, O_2^S)$ or $IT(O_2^S, O_1^S)$ is processed.

FUNC 2. $O_1^S \prec O_2^S$: *boolean, where $O_1 \parallel O_2$.*

```

if  $\beta_{isp}(O_1^S) < \beta_{isp}(O_2^S)$  return true;
else if  $\beta_{isp}(O_1^S) = \beta_{isp}(O_2^S)$ 
  if  $P(O_1^S) < P(O_2^S)$  return true;
  else if  $P(O_1^S) = P(O_2^S)$ 
    if  $T(O_1)=INS$  and  $T(O_2)=DEL$  return true;
    else if  $T(O_1)=T(O_2)=INS$  and  $ID(O_1) < ID(O_2)$ 
      return true;
  return false; //meaning  $O_2^S \prec O_1^S$ 

```

Figure 3: Function to determine if $O_1 \prec O_2$.

4.3 Exclusion Transformation

Given a state S' prior to the current state S , where $S' + SQ = S$ and $SQ = [O_1, O_2, \dots, O_n]$, the established way to compute $\beta_{S'}(O)$, the effect position of operation O relative to S' , is to use ET to exclude the effects of sequence SQ from S . That is, to perform $ET(O^S, [O_n, \dots, O_2, O_1])$. According to [26], exclusively transforming O against a contextually serialized sequence SQ must be processed in reverse order.

Similarly to the IT function in Fig. 2, we define the conceptual ET function in Fig. 4 based on effects relation $\prec$. Now the question is again how to know the effects relation.

FUNC 3. $ET(O^{S^i}, O_i^{S^{i-1}}): O^{S^{i-1}}$, where $S^i = S^{i-1} + O_i^{S^{i-1}}$.

```

 $O^{S^{i-1}} := O^{S^i}$ ;
if  $O_i \prec O$ 
  if  $T(O_i)=INS$ 
     $P(O^{S^{i-1}}) := P(O^{S^{i-1}})-1$ ;
  else //  $T(O_i)=DEL$ 
     $P(O^{S^{i-1}}) := P(O^{S^{i-1}})+1$ ;
return  $O^{S^{i-1}}$ ;

```

Figure 4: ET reinterpreted by $\prec$.

In general some operations in SQ may happen before O and some may be concurrent with it. For those operations in SQ that are concurrent with O , O must have been inclusively transformed against them when we process $ET(O, SQ)$. For example, to do $IT(O_1^{S_3^1}, O_2^{S_3^1})$, we perform $ET(O_1^{S_3^1}, O_3^{S_3^0})$ and $ET(O_2^{S_3^1}, O_3^{S_3^0})$ to compute the effect positions of O_1 and O_2 relative to S^0 . At this point, they both have inclusively transformed against O_3 . Hence we only need to record such a relation when processing inclusion transformations. Then ET can be correctly performed by retrieving the recorded effects relation. In this case it is not necessary to compute their effects relation again.

However, for any operation O_i in SQ such that $O_i \rightarrow O$, there is no recorded relation between them because we do not perform IT between causally-related operations. And we cannot determine the $\prec$ relation between O_i and O by computing their β values relative to a LSP, because our concept of LSP is not defined between causally-related operations. Then for this conceptual ET to work correctly, we have to use the position parameters of operations as far as we can. Let's examine the following three cases.

First consider the case in which O depends on O_i . We say that O_x depends on O_y iff $O_y \rightarrow O_x$ and O_x deletes the character inserted by O_y . As discussed in Sec. 3, relation $\prec$ is not defined between dependent operations. Thus we do not perform $ET(O, O_i)$ in this case. The IT/ET patch in [26] is able to handle this situation but it loses the intuitive physical meaning of ET.

Second consider the case when processing $ET(O^{S^i}, O_i^{S^{i-1}})$, we have $O_i \rightarrow O$, $S^{i-1} + O_i^{S^{i-1}} = S^i$, and S^i is the generation state of O^{S^i} but O does not depend on O_i . In this case, we can unambiguously determine the effects relation between O_i and O by their P parameters only, as defined in Fig. 5. Note to distinguish the operation effects relation defined in Fig. 3, here we annotate it as $\prec_{et}$.

FUNC 4. $O_i^{S^{i-1}} \prec_{et} O^{S^i}$: *boolean*

```

if  $P(O_i^{S^{i-1}}) < P(O^{S^i})$  return true;
else if  $P(O_i^{S^{i-1}}) = P(O^{S^i})$ 
  if  $T(O_i)=T(O)=INS$  return false;
  else if  $T(O_i)=T(O)=DEL$  return true;
  else if  $T(O_i)=DEL$  and  $T(O)=INS$  return false;
return false; //meaning  $O^{S^i} \prec_{et} O_i^{S^{i-1}}$ 

```

Figure 5: Function to partially determine if $O_i \prec_{et} O$.

The condition in Fig. 5, $P(O_i^{S^{i-1}}) = P(O^{S^i})$, $T(O_i)=DEL$ and $T(O)=INS$, handles the case in which a character is deleted by O_i , and then immediately a character is inserted by O at the same position. In absence of semantic information, we assume at the system level either $C(O) \prec C(O_i)$ or $C(O_i) \prec C(O)$ is acceptable if the effect of O_i is excluded. Here we always enforce that $O \prec O_i$ to remove ambiguity.

Third consider the case when processing $ET(O^{S^i}, O_i^{S^{i-1}})$, we have $O_i \rightarrow O$ but S^i is not the generation state of O . To illustrate, suppose three sites start from the same initial state $S^0 = \text{"abcd"}$. Site 1 performs $O_1 = \text{delete("c")}$, 2) and concurrently site 2 performs $O_2 = \text{insert("x", 3)}$. After site 3 receives and executes O_1 , its state becomes $S_3^1 = \text{"abd"}$. When O_2 arrives, because $O_2 \parallel O_1$, it is inclusively transformed against O_1 to be $O_2^{S_3^1} = \text{insert("x", 2)}$. After $O_2^{S_3^1}$ is executed, the state becomes $S_3^2 = \text{"abxd"}$. Then site 3 performs a local operation, $O_3 = \text{insert("y", 3)}$, which changes its state to $S_3^3 = \text{"abxyd"}$. It is clear that the effects relation should be $C(O_1) \prec C(O_2) \prec C(O_3)$.

Now the operation log of site 3 is $[O_1^{S_3^0}, O_2^{S_3^1}, O_3^{S_3^2}]$. To find the effect position of O_3 relative to S^0 , the intuitive way is to exclusively transform O_3 against $O_2^{S_3^1}$ and $O_1^{S_3^0}$ in order, as is shown in Fig. 6. Let $O_3^{S_3^1} = ET(O_3^{S_3^2}, O_2^{S_3^1}) = \text{insert("y", 2)}$. Then we calculate $O_3^{S_3^0} = ET(O_3^{S_3^1}, O_1^{S_3^0})$. The P values of these two operations are tied. The result would be $O_3^{S_3^0} = \text{insert("y", 2)}$, according to the ET functions defined in [26].

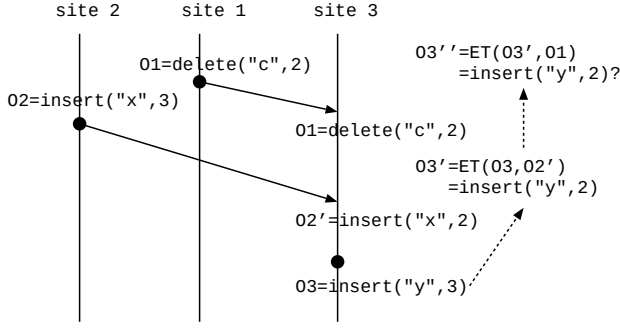


Figure 6: A ERV scenario of ET.

However, according to the above-identified effects relation, the correct result should be $O_3^{S_0} = \text{insert}("y", 3)$.

These three cases reveals the following important fact: Given two contextually serialized operations, $O_i^{S^{i-1}}$ and O^{S^i} , $O_i \rightarrow O$, and O does not depend on O_i , only when S^i is the generation state of O , is it safe to use position parameters to determine the effects relation, and are the original ET functions of [26] equivalent to our conceptual definition of ET in Fig. 4. Otherwise if S^i is not the generation state of O , we cannot always correctly determine their effects relation by directly using the ET functions of [26, 22].

This problem cannot be solved by simply recording the effects relation in IT. As revealed in Fig. 6, due to $O_1 \rightarrow O_3$, O_3 was not inclusively transformed against O_1 before we process $\text{ET}(O_3^{S_1^3}, O_1^{S_1^0})$. Hence there is no recorded information to help remove the ambiguity. For similar reasons, the ET patch to solve the so-called lossy IT problems [22] cannot solve this problem, either. This calls for extra measures in the control algorithm to prevent the problem from happening. Our method to correctly exclude the effects of contextually preceding operations will be presented in Sec. 5.2.

Besides, in our approach, we know $O_1 \prec O_2$ from $\text{IT}(O_2, O_1)$, and $O_2 \prec O_3$ from the observation that O_3 happened after O_2 and the effect position of O_3 is to the right of the character inserted by O_2 . Although it seems possible to transitively infer relation $O_1 \prec O_3$ such that the ERV puzzle in Fig. 6 is easily solved, we suspect that it is generally too expensive to infer relations such as $O_2 \prec O_3$ and $O_1 \prec O_3$ in practice. Hence currently these inferences only serve theoretical analysis purposes and we only record the effects relation between concurrent operations, as in Fig. 7.

5. THE CONTROL ALGORITHM

Our control algorithm partly follows the structure of GOTO [25]. A history buffer (HB) is used at each site to record all operations that have been executed at that site by their order of execution. HB is effectively a sequence of contextually serialized operations, those executed earlier being to the left of those executed later. When a remote operation O is causally-ready for execution, we make a copy of HB in SQ . Suppose there are n operations in SQ , then SQ is transposed such that all operations in the left subsequence $SQ[1, p-1]$ causally precede O , and all operations in the right subsequence $SQ[p, n]$ are concurrent with O . After that, function $\text{SQIT}(O, SQ, p)$ is called to inclusively transform O against $SQ[p, n]$. After the result O' is executed in the current state, O' is appended to HB . In the following we present the part that is different from GOTO.

5.1 Transforming Concurrent Operations

FUNC 5. $\text{SQIT}(O, SQ, p): O'$

```

1   $O' := O;$ 
2  for  $i = p$  to  $n$  do
3     $SQ_{lsp} := \text{ComputeLSP}(O', i, SQ);$ 
4     $SD := \text{BuildSD}(SQ_{lsp});$ 
5     $\langle \beta_{lsp}(O'), \delta_{lsp}(O') \rangle := \text{Compute}\beta\delta(O', SD);$ 
6     $\langle \beta_{lsp}(SQ[i]), \delta_{lsp}(SQ[i]) \rangle := \text{Compute}\beta\delta(SQ[i], SD);$ 
7    Determine and store the effects relation of  $O$  and  $SQ[i]$ ;
8     $O' := \text{IT}(O', SQ[i]);$ 
9  return  $O'$ ;

```

Figure 7: The SQIT algorithm

As shown in Fig. 7, algorithm SQIT inclusively transforms a given operation O against all operations in SQ that are concurrent with O . For the given index p , it is assumed that all operations in subsequence $SQ[1, p-1]$ causally precedes O and those in $SQ[p, n]$ are concurrent with O . For each $SQ[i] \parallel O$, we call algorithm ComputeLSP to get the subsequence of operations between their last synchronization point (LSP) and the current state. The result, SQ_{lsp} , is the sequence of operations that we need to exclude from O and $SQ[i]$ for determining their effect positions relative to LSP. Then we build a special sequence SD based on SQ_{lsp} , which is used next to compute $\beta_{lsp}(O)$ and $\beta_{lsp}(SQ[i])$. Base on these β values, we determine the effects relation $\prec$ between O and $SQ[i]$ as in Fig. 3 and save this relation for future uses. Then we perform $\text{IT}(O, SQ[i])$ based on relation $\prec$ as in Fig. 2. It can be shown that the position parameters of O and $SQ[i]$ here are defined on the same state and their definition context is $SQ[1, i-1]$.

FUNC 6. $\text{ComputeLSP}(O, j, SQ): SQ_{lsp}$

```

1   $V_{lsp} := \text{MIN}(V(O), V(SQ[j]));$ 
2  if  $\exists r (V(SQ[r]) < V_{lsp} \text{ and } \forall O_s \in SQ[r+1, n] \Rightarrow V(O_s) \not< V_{lsp})$ 
3     $\langle SQ_t, i \rangle := \text{TransposeL2R}(V_{lsp}, SQ[1, r]);$ 
4     $SQ_t := SQ_t + SQ[r+1, n];$ 
5     $SQ_{lsp} := SQ_t[i, j-1];$ 
6  else //  $\nexists r: V(SQ[r]) < V_{lsp}$ 
7     $SQ_{lsp} := SQ[1, j-1];$ 
8  return  $SQ_{lsp};$ 

```

Figure 8: The ComputeLSP algorithm

In algorithm ComputeLSP (Fig. 8), operation O is to be inclusively transformed against $SQ[j]$. Line 1 computes V_{lsp} , the state vector identifying the LSP of O and $SQ[j]$, by calling function MIN to get the minimum value of each corresponding element of the state vectors of O and $SQ[j]$. Function $V(O)$ returns the state vector of operation O . Given any two state vectors V_1 and V_2 of the same length, $V_1 < V_2$ iff $\forall i: V_1[i] \leq V_2[i]$ and $\exists k: V_1[k] < V_2[k]$.

Then we find the subsequence in SQ that will be used for building the special sequence SD . As shown in line 2, we scan SQ from right to left to find the first operation $SQ[r]$ such that its state vector is less than V_{lsp} and the state vectors of all operations in $SQ[r+1, n]$ are no less than V_{lsp} . If such an operation $SQ[r]$ is not found (lines 6-7), the state vectors of all operations in $SQ[1, n]$ must be no less than V_{lsp} , which means that LSP is the initial state. Hence we return subsequence $SQ[1, j-1]$.

If $SQ[r]$ exists (lines 2-5), due to causality preservation,

we must have $SQ[r] \rightarrow SQ[j]$ and $r < j$. There may still be operations in the subsequence $SQ[1, r]$ whose state vectors are no less than V_{isp} . Hence we call algorithm TransposeL2R to transpose $SQ[1, r]$ such that all operations whose state vectors are less than V_{isp} are moved to the left of those whose state vectors are no less than V_{isp} . Let the transposed result be SQ_t and i point to the leftmost operation in SQ_t whose state vector is no less than V_{isp} . After concatenating the SQ_t and $SQ[r+1, n]$, the new sequence has the property that the state vectors of all operations in $SQ[i, n]$ are no less than V_{isp} , and the state vectors of all operations in $SQ[1, i-1]$ are less than V_{isp} . Hence subsequence $SQ[i, j-1]$ is returned.

FUNC 7. *TransposeL2R*(V, SQ): $\langle SQ_r, r \rangle$

```

1   $SQ_r := SQ$ ;
2   $u := n$ ;
3  for  $i = n-1$  to 1 do
4    if  $V(SQ_r[i]) \not\prec V$ 
5       $SQ_t := \text{L2RIT}(SQ_r[i, u])$ ;
6       $SQ_r := SQ_r[1, i-1] + SQ_t + SQ_r[u+1, n]$ ;
7       $u := u-1$ ;
8  return  $\langle SQ_r, u+1 \rangle$ ;
```

Figure 9: The TransposeL2R algorithm

Algorithm TransposeL2R (Fig. 9) transposes a given contextually serialized sequence such that all operations whose state vectors are less than V are moved to the left of those whose state vectors are no less than V . Note it is assumed in the beginning of this algorithm that the rightmost operation in SQ has a state vector less than V , according to algorithm ComputeLSP. Assume that SQ has n operations. We scan SQ from right to left, every time an operation $SQ[i]$ with a state vector no less than V is found, we call algorithm L2RIT to transpose it to the rightmost position of sequence $SQ[i, u]$. Index u is maintained to always point to the leftmost operation with a state vector less than V .

FUNC 8. *L2RIT*(SQ): SQ'

```

1   $SQ' := SQ$ ;
2   $O_{i-1} := SQ'[1]$ ;
3  for  $i = 2$  to  $n$  do
4     $O_i := \text{ET}(SQ'[i], O_{i-1})$ ;
5     $O_{i-1} := \text{IT}(O_{i-1}, O_i)$ ;
6   $SQ' := SQ'[1, i-2] + [O_i, O_{i-1}] + SQ'[i+1, n]$ ;
7  return  $SQ'$ ;
```

Figure 10: The L2RIT algorithm

Given an operation sequence SQ of length n , algorithm L2RIT (Fig. 10) is responsible for transposing $SQ[1]$ to the rightmost side of SQ . Based on algorithm TransposeL2R, we can assume here that only the state vector of $SQ[1]$ is no less than V and that the state vectors of all operations in $SQ[2, n]$ are less than V . In the loop of lines 3-6, at each step we transpose O_{i-1} and O_i by calling ET and IT. Similar to GOTO [25, 22], it can be shown that IT/ET here are all performed between concurrent operations that have been inclusively transformed before. Hence the effects relation is available and IT/ET can be correctly processed.

5.2 Excluding Operation Effects

Now we present algorithms BuildSD and Compute $\beta\delta$ that are used in Fig. 7. We first introduce a new concept state

difference. Based on this concept, we will be able to reduce the problem of excluding the effects of a given sequence SQ into the problem of excluding the effects of its equivalent state difference. Properties of the state difference will make it safe to compute the β value of a given operation relative to a previous state with our definition of ET in Sec. 4.3.

Suppose a contextually serialized sequence $SQ = [O_1^{S^0}, O_2^{S^1}, \dots, O_n^{S^{n-1}}]$ is executed in a state S^0 and $S^0 + SQ = S^n$. The state difference SD between S^0 and S^n is the net effects of SQ such that $S^0 + SD = S^n$ or $SQ \equiv SD$. Conceptually SD captures the increase and decrease of content in S^0 that are caused by the execution of SQ . Note operations in a state difference are stringwise. We use function $S(O)$ to represent the string content that O inserts or deletes, and function $L(O)$ to denote the length of $S(O)$.

5.2.1 SD Construction and Properties

A state difference is incrementally constructed by calling the algorithm BuildSD defined in Fig. 11.

FUNC 9. *BuildSD*(SQ): SD

```

1   $SD := \{SQ[1]\}$ ;
2  for ( $i = 2$ ;  $i \leq n$ ;  $i++$ )
3     $\langle \beta_{S^0}(SQ[i]), \delta_{S^0}(SQ[i]) \rangle := \text{Compute}\beta\delta(SQ[i], SD)$ ;
4    merge  $SQ[i]$  into  $SD$ ;
5  return  $SD$ ;
```

Figure 11: To incrementally build a state difference.

We first initialize SD to $SQ[1]$. Apparently the position parameter of $SQ[1]$ is relative to S^0 , that is, $\beta_{S^0}(SQ[1]) = P(SQ[1])$. Then we incrementally incorporate the effects of other operations in SQ into the partially built SD . For any $SQ[i] \in SQ$, we compute the effect position of $SQ[i]$ relative to S^0 by calling algorithm Compute $\beta\delta$. Then we merge $SQ[i]$ into SD according to its β and δ values:

1. First locate an insert operation $O_j \in SD$ such that $\beta(O_j) = \beta(SQ[i])$. If such a O_j is found, it means that $SQ[i]$ and O_j intend to insert or delete characters at the same or adjacent positions relative to S^0 . We insert $C(SQ[i])$ into string $S(O_j)$ if $T(SQ[i]) = \text{INS}$, or delete $C(SQ[i])$ from $S(O_j)$ if $T(SQ[i]) = \text{DEL}$, according to the value of $\delta(SQ[i])$ which points to the position of insert or delete in $S(O_j)$.
2. Otherwise if O_j is not found, we add $SQ[i]$ into SD by the following rule: For any pair of operations O_x and O_y in SD , O_x precedes O_y iff (1) $\beta(O_x) < \beta(O_y)$, or (2) $\beta(O_x) = \beta(O_y)$ but $T(O_x) = \text{INS}$ and $T(O_y) = \text{DEL}$. Every time a new operation O_i is added into SD , its position parameter is reset to its β value.

As a result of this merging process, SD has the following interesting properties: (1) It does not include inverse operations that counteract the effects of each other and generate no impact on S^0 . (2) Delete operations are still characterwise. (3) For any operation $O \in SD$, $P(O) = \beta_{S^0}(O)$. (4) Operations are ordered ascendingly by their effect positions that are relative to S^0 , or insert precedes delete if there is a tie. Due to the effects relation $\prec$ defined in Sec. 4.1, it is easy to show that operations in SD are exactly ordered by their effects relation, i.e., O_x precedes O_y iff $O_x \prec O_y$. This is because all operations in SD can be considered as being

concurrent to each other and S^0 can be considered as the LSP of all operations in SD .

5.2.2 Implications of SD Properties

Once a state difference SD is constructed as above, we use algorithm $\text{Compute}\beta\delta$ to compute the effect position of operation O^{S^n} relative to S^0 , by excluding the effects of operations in SD . Due to $SD \equiv SQ$, excluding the effects of operations in SD from O^{S^n} will serve the same purpose of excluding effects of operations in SQ . Since our purpose is to get $\beta(O)$ relative to S^0 , we will not always need ET or can use it differently. Hence the problems of ET discussed in Sec. 4.3 can be avoided.

We introduce notation SQ^{-1} to denote the sequence obtained by reversing the ordering of operations in sequence SQ . Due to the above properties of SD , we may view SD as a special sequence. Then the sequence SD^{-1} is of particular interest. Given any pair of operations $SD^{-1}[i]$ and $SD^{-1}[i+1]$, their position parameters are defined relative to S^0 , and $SD^{-1}[i+1] \prec SD^{-1}[i]$. Hence the execution of $SD^{-1}[i]$ in S^0 does not change the definition context of $SD^{-1}[i+1]$ or the semantics of $P(SD^{-1}[i+1])$.

In other words, we can safely execute $SD^{-1}[1]$ in state S^0 , then execute $SD^{-1}[2]$ in the state that results, and so on, until $SD^{-1}[m]$. Suppose m is the number of operations in SD . It is apparent that the state resulted from the execution of $SD^{-1}[m]$, or $SD[1]$, is but S^n , which is exactly the definition state of O . Then we can safely translate the problem of excluding the effects of SD from O a step further into the problem of excluding the effects of SD^{-1} from O . According to [26], the effects of operations in a contextually serialized sequence must be excluded in reverse order. That is, we should perform $\text{ET}(O, (SD^{-1})^{-1})$, which is nothing but $\text{ET}(O, SD)$. This justifies that in algorithm $\text{Compute}\beta\delta$ we only need to exclude the effects of operations in SD in the order of $SD[1]$ to $SD[m]$.

5.2.3 Computing β from SD

Due to space reasons, here we only describe how algorithm $\text{Compute}\beta\delta$ works. The algorithm in full detail is given in [14]. Note the ET function to be used here follows the definition in Fig. 4, where the $\prec$ relation is determined as in Fig. 5, if it is not available as a result of some previous IT as in the SQIT algorithm (Fig. 7).

Algorithm $\text{Compute}\beta\delta$ works as follows. We exclusively transform O against operations in SD in order from $SD[1]$ to $SD[m]$, until some operation $SD[i]$, $1 \leq i \leq m$, satisfies one of the following three conditions:

- (1) $P(O) < P(SD[i])$,
- (2) $T(SD[i]) = \text{INS}$ and $P(SD[i]) \leq P(O) < P(SD[i]) + L(SD[i])$,
- (3) $T(SD[i]) = T(O) = \text{INS}$ and $P(O) = P(SD[i]) + L(SD[i])$.

Condition (1) means the effect position of $SD[i]$ goes to the right of the effect position of O . Due to the ordering of operations in SD , for any operation $SD[j] \in SD[i, m]$, we have $P(SD[j]) > P(O)$. Then performing $\text{ET}(O, SD[j])$ further will no longer affect the result. Note at this point we have excluded the effects of all operations in $SD[1, i-1]$ from O . As a result, we return the effect of O relative to S^0 as $\beta_{S^0}(O) = P(O)$ and $\delta_{S^0}(O) = 0$.

Under condition (2), the effect position of O falls into the range of a string inserted by $SD[i]$. Note O is a character-wise operation. It means that O deletes a character from

$S(SD[i])$ or inserts a character into $S(SD[i])$. In the former case, O deletes a character that was inserted by some operation in SQ say $SQ[j]$. According to our definition in Sec. 4.3, we know that O depends on $SD[i]$, or more accurately, $SQ[j]$. In the latter case, O inserts a character together with some other insert operations in SQ at the same position $P(SD[i])$ relative to S^0 . Either way we know the effect position of O relative to S^0 will be the same as $P(SD[i])$. Hence there is no need to perform ET to get $\beta_{S^0}(O)$ nor is it necessary to examine $SD[i+1, m]$ further. As a result, we return $\beta_{S^0}(O) = P(SD[i])$, and $\delta_{S^0}(O) = P(O) - P(SD[i])$ if $T(O) = \text{INS}$, or $\delta_{S^0}(O) = P(O) - P(SD[i]) + 1$ if $T(O) = \text{DEL}$. Apparently the value of $\delta_{S^0}(O)$ is relative to string $S(SD[i])$.

Condition (3) means O inserts a character right after the string inserted by $SD[i]$. It cannot be processed by simply excluding the effect of $SD[i]$ without considering the effect of $SD[i+1]$. This is illustrated by the following example. Suppose $S^0 = "abcd"$, $SD = [SD_i^{S^1}, SD_{i+1}^{S^1}]$, $SD_i^{S^1} = \text{insert}("xy", 2)$, $S^0 + SD = S^2$, and $O^{S^2} = \text{insert}("z", 4)$. Then what is $\beta_{S^0}(O^{S^2})$? There are two possibilities: character 'z' is inserted either immediately before or after the landmark character 'c'. In the first possibility, $\beta_{S^0}(O) = 2$ and $\delta_{S^0}(O) = 2$, while in the second, $\beta_{S^0}(O) = 3$ and $\delta_{S^0}(O) = 0$. The result depends on the operation $SD_{i+1}^{S^0}$.

Due to the properties of state difference, if $SD_{i+1}^{S^0}$ is an insertion, then $P(SD_{i+1}^{S^0}) > P(SD_i^{S^1})$ must hold; or if $SD_{i+1}^{S^0}$ is a deletion but $P(SD_{i+1}^{S^0}) \neq P(SD_i^{S^1})$, then $P(SD_{i+1}^{S^0}) > P(SD_i^{S^1})$ must hold. Either way $SD_{i+1}^{S^0}$ will not impact $P(O^{S^0})$, resulting that 'z' is inserted immediately before 'c', that is, 'z' will be appended to $S(SD_i)$. Hence we return $\beta_{S^0}(O) = P(SD_i)$ and $\delta_{S^0}(O) = P(O) - P(SD_i)$.

If $SD_{i+1}^{S^0} = \text{delete}("c", 2)$, or generally $T(SD_{i+1}) = \text{DEL}$ and $P(SD_{i+1}) = P(SD_i)$, we first compute $O^{S^1} = \text{ET}(O^{S^2}, SD_i^{S^1})$ and get $O^{S^1} = \text{insert}("z", 2)$. Then compute $O^{S^0} = \text{ET}(O^{S^1}, SD_{i+1}^{S^0})$. Note here because $SD_{i+1}^{S^0}$ is a deletion that survived the merging process of algorithm BuildSD, it must be a characterwise operation and uniquely correspond to (and bear the same timestamp of) a deletion in the original SQ .

There are two possibilities: First, if $O \parallel SD_{i+1}$, then O must have been inclusively transformed against SD_{i+1} . The result of ET can then be unambiguously computed by checking the recorded effects relation between SD_{i+1} and O . Second, if $SD_{i+1} \rightarrow O$, because $P(O^{S^1}) = P(SD_{i+1}^{S^0})$, $T(SD_{i+1}) = \text{DEL}$ and $T(O) = \text{INS}$, the result of ET is not unique but the ambiguity can be resolved in our ET definition in Fig. 4 and Fig. 5.

Depending on the outcome of O^{S^0} , if it is to insert a character at the same position as SD_i , which is immediately before $C(SD_{i+1})$, assuming it were not deleted by SD_{i+1} , then $C(O)$ should be merged into $S(SD_i)$. Hence we return $\beta_{S^0}(O) = P(SD_i)$ and $\delta_{S^0}(O) = P(O) - P(SD_i)$. Otherwise, O^{S^0} is to insert a character right after $C(SD_{i+1})$, if it were not deleted by SD_{i+1} . Then the main loop continues if there are still more operations in $SD[i+2, m]$. Note the lookahead operation SD_{i+1} is skipped since its effect has already been excluded from O . $\square$

From the above algorithm description, we can see that only delete operations in SD need to carry timestamps and insertions do not need timestamps. δ values computed here

are only used in algorithm BuildSD. We explain in [14] why it is not necessary to use δ values in Fig. 3 for determining the effects relation $\prec$ of operations.

6. RELATED WORK

The TP2 puzzle has been approached from two different directions. One is to extend the original control algorithm of [7] such that there is only one transformation path at all sites and thus TP2 is no longer a necessary condition of convergence, as in GOT [26], NICE [19], and SOCT3/4 [27]. The second direction is to extend the original IT functions of [7] such that they are able to satisfy the TP2 condition, as in SOCT2 [20] and IMOR [11].²

NICE uses a notification server so that clients take turns to propagate operations through the server. SOCT3/4 uses a sequencer to maintain a global total order of execution. Our solution to the convergence problem is more general and applicable in peer-to-peer and mobile computing environments in which a central component may not be present. GOT enforces a global total order of execution and is also able to achieve convergence in a fully replicated system. Nonetheless, NICE and SOCT3/4 cannot ensure operation effects relation. For example, at site 3 of Fig. 1, both NICE and SOCT3/4 will generate the same wrong result “a12c”. GOT does not suffer from the ERV puzzle in Fig. 1 but fails in other more complicated ERV scenarios [14].

Algorithms adOPTed [18] and GOTO [25, 22] essentially inherits the IT functions of dOPT [7]. Hence they assume TP2 and cannot achieve convergence without extra measures as in [26, 19, 27]. The problems with ET as discussed in Sec. 4 are avoided in GOTO because it only performs IT/ET between concurrent operations [22]. According to [23], a solution to the ERV (or “false-tie” in their term) problem has been implemented in [24] but not published yet. The solution was not included in [26, 19, 25, 22]. Our approach is independent and different. Work in [15] takes a similar idea but it does not address the ERV problem and its notion of consistency still follows that of [26].

SOCT2 introduces two new parameters to the insert operation $insert(p, c, b, a)$: b is the set of operations that have deleted characters before the inserted character, and a is the set of operations that have deleted characters after the inserted character. Due to the lack of critical detail in [20], however, it is not clear how b and a are computed. We suspect that they are derived by using “backward transposition”, which is essentially the same as ET in [26]. However, due to our analysis in Section 4.3, backward transposition (or ET) fails in some scenarios. Unfortunately, this problem was not addressed in previous work including [20].

IMOR simplifies SOCT2 by using one new parameter in the insert operation $insert(p, c, ip)$, where ip is the position of the operation relative to its generation state. When inclusively transforming two concurrent (insert) operations, if their position parameters equal, then their ip values are compared, after which the ASCII codes of the characters are compared to break the tie. Intuitively, it may not make sense to compare the ip values of two concurrent operations if they are not defined on the same state. The nature of this problem resembles that of the dOPT puzzle [25].

The counterexample in Fig. 12 shows that IMOR is not

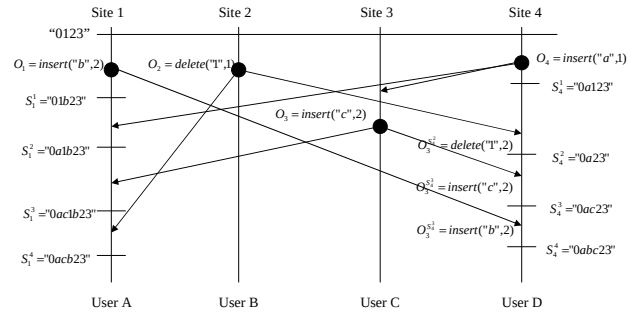


Figure 12: A divergence and ERV scenario of IMOR.

able to solve the ERV problem nor the divergence problem. Suppose all the four sites in Fig. 12 start with the same document state $S^0 = "0123"$. User A performs $O_1 = insert("b", 2)$ at site 1. Concurrently, user B performs $O_2 = delete("1", 1)$ at site 2, and the user D performs $O_4 = insert("a", 1)$ at site 4. After O_4 arrives at site 3 and is executed, user C performs $O_3 = insert("c", 2)$ at site 3. First consider site 4. Assume that O_2 arrives first. After processing $IT(O_2, O_4)$, $O_2^{S_4^1} = delete("1", 2)$ and $S_4^0 + [O_4] = S_4^1$. Similarly, when O_3 arrives, we get $O_3^{S_4^1} = IT(O_3, O_2^{S_4^1}) = insert("c", 2)$ and $S_4^1 + [O_3^{S_4^1}] = S_4^2$. Finally, O_1 is received. After inclusively transforming O_1 against $[O_4, O_2^{S_4^1}]$, we get $O_1^{S_4^2} = insert("b", 2)$. Then what will be the result of $IT(O_1^{S_4^2}, O_3^{S_4^2})$? Due to the IT functions defined in [11], since $P(O_1^{S_4^2}) = P(O_3^{S_4^2})$ and their original execution positions ($IP(O_1) = 2$ and $IP(O_3) = 2$) are also the same, then we compare the character code to break the tie. As a result, we have $O_1^{S_4^3} = insert("b", 2)$. After it is executed, the state of site 4 becomes $S_4^4 = "0abc23"$. This result is not correct, because ‘b’ should be after ‘c’ in position due to the landmark character ‘1’.

Because IMOR fails to solve the ERV problem, it is also not able to ensure convergence. Consider site 1 in Fig. 12. After O_1 is executed, we have $S_1^1 = "01b23"$. When O_4 arrives, $O_4^{S_1^1} = IT(O_4, O_1) = insert("a", 1)$, after the execution of which we get $S_1^2 = "0a1b23"$. Then O_3 arrives. The history buffer of site 1 is $[O_1^{S_1^0}, O_4^{S_1^1}]$. We have $(O_4 \rightarrow O_3) \parallel O_1$. It does not make sense to inclusively transform O_3 against O_1 (by directly using their IT functions) because their position parameters are relative to different states. This is a typical dOPT puzzle scenario. Although it is not clear how the control algorithm of IMOR works due to the absence of critical detail in [11], it is fair to assume that IMOR is able to correctly solve the dOPT puzzle, e.g., if it inherits the GOTO-like control algorithm of SOCT2. The correct executable form of O_3 should be $O_3^{S_1^2} = insert("c", 2)$. After it is executed, we get $S_1^3 = "0ac1b23"$. Now O_2 arrives. It is concurrent to all operations currently in the history buffer of site 1. No matter how they are ordered, due to the IT functions of IMOR, we get $O_2^{S_1^3} = delete("1", 3)$. After it is executed, the final state of site 1 becomes $S_1^4 = "0acb23"$, which is obviously divergent from state $S_4^4 = "0abc23"$.

²The approach of [11] was not named. Here we name it after its authors as IMOR for convenience.

7. CONCLUSIONS

This work solves two inadequately addressed problems in group editors. While our solution to the convergence problem in the presence of arbitrary transformation paths is a more general alternative to [26, 19, 27], our solution to the ERV problem finds no pioneer in the literature. Our notion of CSM consistency is a more realistic reflection on the role of OT in consistency maintenance and brings new insights in interpreting OT algorithms. It can be shown that our approach works correctly in the scenarios of Fig. 1, 6, 12.

For space reasons, a detailed presentation of this work is given in [14]. It includes a 12-page formal correctness proof of that our ET is unambiguous, that our IT satisfies TP1 and TP2, and that our approach achieves CSM consistency. We plan to address performance issues and extend this work to tree based structures in future research. Further information is available at <http://faculty.cs.tamu.edu/lidu/>.

8. REFERENCES

- [1] P. Bellini, P. Nesi, and M. B. Spinu. Cooperative visual manipulation of music notation. *ACM Transactions on Computer-Human Interaction*, 9(3):194–237, Sept. 2002.
- [2] R. Bentley and P. Dourish. Medium versus mechanism: Supporting collaboration through customization. In *ECSCW'95 Proceedings*, 1995.
- [3] A. H. Davis, C. Sun, and J. Lu. Generalizing operational transformation to the standard general markup language. In *ACM CSCW'02*, pages 58–67, Nov. 2002.
- [4] P. Dewan, R. Choudhary, and H. Shen. An editing-based characterization of the design space of collaborative applications. *Journal of Organizational Computing*, 4(3):219–240, 1994.
- [5] P. Dourish. Consistency guarantees: Exploiting application semantics for consistency management in a collaboration toolkit. In *ACM CSCW'96 Proceedings*, pages 268–277, Cambridge, MA, Nov. 1996.
- [6] W. K. Edwards. Policies and roles in collaborative applications. In *ACM CSCW'96 Proceedings*, pages 11–20, 1996.
- [7] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *ACM SIGMOD'89 Proceedings*, pages 399–407, Portland Oregon, 1989.
- [8] S. Greenberg and D. Marwood. Real-time groupware as a distributed system: Concurrency control and its effect on the interface. In *ACM CSCW'94 Proceedings*, pages 207–217, Chapel Hill, NC, Oct. 1994.
- [9] J. Grudin. Why CSCW applications fail: Problems in the design and evaluation of organizational interfaces. In *ACM CSCW'88 Proceedings*, pages 85–93, Portland, Oregon, Sept. 1988.
- [10] C. M. Hymes and G. M. Olson. Unblocking brainstorming through the use of simple group editor. In *ACM CSCW'92 Proceedings*, pages 99–106, Nov. 1992.
- [11] A. Imine, P. Molli, G. Oster, and M. Rusinowitch. Proving correctness of transformation functions in real-time groupware. In *European CSCW'03 Proceedings*, Sept. 2003.
- [12] M. J. Knister and A. Prakash. A framework for undoing actions in collaborative systems. *ACM Transactions on Computer-Human Interaction*, 1(4):295–330, Dec. 1994.
- [13] D. Li and R. Li. Transparent sharing and interoperation of heterogeneous single-user applications. In *ACM CSCW'02 Proceedings*, pages 246–255, Nov. 2002.
- [14] D. Li and R. Li. Ensuring consistency in real-time group editors. *ACM Transactions on Computer-Human Interaction*, Apr. 2004. Under review.
- [15] D. Li and R. Li. Ensuring content and intention consistency in real-time group editors. In *Proceedings of the 24th IEEE International Conference on Distributed Computing Systems (ICDCS'04)*, pages 748–755, Mar. 2004.
- [16] C. M. Neuwirth, D. S. Kaufer, R. Chandhok, and J. H. Morris. Computer support for distributed collaborative writing: defining parameters of interaction. In *Proceedings of the ACM conference on Computer supported cooperative work*, pages 145–152, Chapel Hill, NC, Oct. 1994.
- [17] S. Noel and J.-M. Robert. Empirical study on collaborative writing: What do co-authors do, use, and like. *Journal of Computer Supported Cooperative Work*, 13:63–89, 2004.
- [18] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhauser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *ACM CSCW'96 Proceedings*, pages 288–297, Nov. 1996.
- [19] H. Shen and C. Sun. Flexible notification for collaborative systems. In *ACM CSCW'02*, pages 77–86, Nov. 2002.
- [20] M. Suleiman, M. Cart, and J. Ferrié. Concurrent operations in a distributed and mobile collaborative environment. In *Proceedings of the IEEE Fourteenth International Conference on Data Engineering*, pages 36–45, Orlando, Florida, USA, Feb. 1998.
- [21] C. Sun. Optional and responsive fine-grain locking in Internet-based collaborative systems. *IEEE Transactions on Parallel and Distributed Systems*, 13(9):994–1008, Sept. 2002.
- [22] C. Sun. Undo as concurrent inverse in group editors. *ACM Transactions on Computer-Human Interaction*, 9(4):309–361, Dec. 2002.
- [23] C. Sun. False-tie puzzle solved in reduce and cword group editors. Personal Communication, Feb. 2004.
- [24] C. Sun. The CoWord and CoPowerpoint Projects. <http://reduce.qpsf.edu.au/coword/>, 2004.
- [25] C. Sun and C. Ellis. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *ACM CSCW'98*, pages 59–68, Dec. 1998.
- [26] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, Mar. 1998.
- [27] N. Vidot, M. Cart, J. Ferrie, and M. Suleiman. Copies convergence in a distributed realtime collaborative environment. In *ACM CSCW'00*, pages 171–180, Dec. 2000.